

第3章 数组与指针

数组和指针是 C++ 中两个重要的概念。在程序设计中需要处理批量数据时，一般都使用数组。数组是具有一定顺序关系的若干变量的集合。

指针是保存变量地址的一种变量。正确和灵活地使用指针，可以使设计出来的程序简洁、高效，但指针使用不当，又会造成危险。因此在学习时要注意弄清概念，掌握指针的真正含义，并在程序设计中会使用指针这一重要工具。

通过本章的学习，要求掌握数组和指针的概念和使用方法。理解指针和数组之间的关系以及它们的用法。

3.1 数组

数组是类型相同、数目固定的若干个变量的有序集合。数组中的每个变量称为数组元素。数组中的每个元素对应存储器中的一个存储空间，元素的值就存放在这个空间，并用下标标识数组中每个元素的位置。依据元素排列方式的不同，数组分为一维数组、二维数组等。数组在实际应用中很广泛。

3.1.1 数组的定义及元素引用

数组是由相同数据类型的元素按一定规则组合而成的，是一组具有相同名字和不同下标的变量。例如：

```
int a[10]
```

中的 `a` 为数组名，方括号中的 `10` 为元素个数。数组名后有一个方括号时是一维数组，有两个方括号时是二维数组，依此类推。常用的是一维和二维数组。例如：

```
int a[10], b[3][4], x[5][4][6]
```

其中，`a` 为一维数组；`b` 为二维数组；`x` 为三维数组。数组名由用户定义。数组与其他变量一样，在使用之前必须定义。因为数组中的所有元素都具有相同的类型，所以定义数组与定义变量一样，用类型名定义。

1. 一维数组的定义及元素引用

(1) 一维数组的定义

定义数组包括定义数组的类型、名字、维数及数组中元素的个数。一维数组的定义形式为：

```
类型名 数组名[常量表达式]
```

其中常量表达式值为数组元素的个数。例如：

```
int a[5];
```

```
char ch[10];
```

```
double data[15];
```

以上定义了三个数组，其中数组名为 `a` 的数组共有 5 个元素，每个元素中可以存放一整数；数组 `ch` 中有 10 个元素，每个元素中存放一字符型数据。数组 `data` 中有 15 个元素，每个元素中存放一双精度型数据。

定义数组后编译系统就为其分配一块连续的存储单元。所谓连续的区域指相邻两元素之间没有空间。例如，对上述 `a` 数组将分配一块连续的区域，用于存放 `a` 数组中的 5 个整型数；为 `ch` 数组分配一块能存放 10 个字符的连续区域；为 `data` 数组分配一块能存放 15 个双精度型数据的连续区域。

注意，定义数组时“`[]`”中必须是整型常量表达式，不能是变量。因为编译系统必须确

切知道保存这个数组需要的存储空间的大小。

(2)元素引用

定义数组后才能在程序中使用。在程序中使用的是数组的元素。一维数组元素的引用形式为:

数组名[下标]

C++规定数组元素下标的取值由 0 开始,最后一个元素的下标是数组定义中常量表达式的值减 1。例如:

```
int a[5];
```

中 5 个元素分别是 a[0]、a[1]、a[2]、a[3]、a[4]。

又如:

```
char ch[10];
```

中 10 个元素分别是 ch[0]、ch[1]、ch[2]、……、ch[9]。

在程序中使用数组元素与使用变量一样,例如:

```
int a[5];
```

```
a[0]=12;           ?//给 a[0]元素赋值 12
```

```
a[1]=30;
```

```
a[2]=a[0]+a[1];     //将 a[0]和 a[1]元素值相加结果存 a[2]
```

数组的下标必须是整型表达式。在程序中一般常用一个变量来控制数组元素的下标,依靠变量的变化表示不同的数组元素。例如:

```
int a[5];
```

```
for (int i=0; i<5; i++)
```

```
    a[i]=2*i;
```

2.二维数组的定义及元素引用

二维以上的数组称为多维数组,常用的是二维数组。二维数组的定义形式为:

类型名 数组名[常量表达式 1][常量表达式 2]

例如:

```
int x[3][4];
```

定义了一个二维数组,数组名为 x, [3]为第 1 维, [4]为第 2 维。可以将二维数组视为一矩阵。[常量表达式 1]表示矩阵的行, [常量表达式 2]表示矩阵的列。因此,该数组表示的是一个 3 行 4 列的矩阵。行和列下标元素的取值也是由 0 开始,因此上述 x 数组元素形成的矩阵为:

```
x[0][0] x[0][1] x[0][2] x[0][3]——0 行
```

```
x[1][0] x[1][1] x[1][2] x[1][3]——1 行
```

```
x[2][0] x[2][1] x[2][2] x[2][3]——2 行
```

与一维数组相同,在程序中使用的是二维数组中的元素。二维数组元素的引用形式为:

数组名[行下标][列下标]

例如, x[0][0]、x[1][2]等都是二维数组元素。

二维数组在计算机内是按行号升序、行内按列号升序存放的。即在一连续的空间内先存放第 0 行,接着存放第 1 行……。在程序中一般常用两个变量来控制二维数组元素的行下标和列下标。例如:

```
double data[5][4];
```

```
for (int i=0; i<5; i++)
```

```
    for (int j=0; j<4; j++)
```

```
        data[i][j]=(i+j)*0.5;
```

3.1.2 数组的初始化

在定义数组时可以被赋予初值，称为数组的初始化。

1.一维数组的初始化

例如：

```
int a[5]={5, 10, 15, 20, 25};
```

表示定义 a 数组的同时给 a 数组赋初值。初始化值为:a[0]=5, a[1]=10, a[2]=15, a[3]=20, a[4]=25。一维数组初始化时，数组名后[]中的常量可以省略。例如：

```
int ax[]={85, 10, 75, 40, 25, 35};
```

编译系统根据{}中所赋初值的个数自动计算数组的大小。以上 ax 数组自动定义成有 6 个元素。给数值型数组赋初值时，若所赋常量的个数少于定义数组元素的个数时，没有赋值的元素值按 0 处理。例如：

```
int a[5]={1, 2, 3};
```

初始化后，a 数组中 5 个元素的值为:a[0]=1, a[1]=2, a[2]=3, a[3]=0, a[4]=0。反之，若所赋初值多于定义数组元素个数时，则产生编译错误。

例 3.1 求 10 个数的总和及平均值。

```
#include <iostream.h>
void main()
{ int array[10]={65,87,90,80,84,85,53,46,95,70};
  int sum(0), average;
  for (int i=0; i<10; i++) sum+=array[i];
  average=sum/10;
  cout<<" 总和=" <<sum<<endl;
  cout<<" 平均值=" <<average<<endl;
}
```

运行结果为：

总和=755

平均值=75

2.二维数组的初始化

二维数组初始化有以下几种形式，例如：

```
int b[2][3]={{1, 3, 5}, {2, 4, 6}};
```

初始化后形成的矩阵为：

1 3 5——0 行

2 4 6——1 行

该数组也可以用以下形式初始化：

```
int b[2][3]={1, 3, 5, 2, 4, 6};
```

程序执行时先取{}中前 3 个数为第 0 行，再取后 3 个数为第 1 行。

同样，二维数组初始化时也可以对部分元素赋初值，数值型数组没有赋值的元素自动按 0 值处理。例如：

```
int x[3][3]={{1}, {2, 3}, {4, 5, 6}};
```

赋初值后形成的矩阵为：

1 0 0

2 3 0

4 5 6

若写成

```
int x[3][3]={1, 2, 3, 4, 5, 6, 7};
```

则形成的矩阵为:

```
1  2  3
4  5  6
7  0  0
```

初始化时取前 3 个作为第 0 行, 再取 3 个作为第 1 行, 后面不足 3 个的补 0。

二维数组对全部元素赋初值时, 定义中最左边的[]中常量值可以省略, 编译系统根据初始化时所赋值的个数自动计算[]中的值。例如:

```
int x[][4]={1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12};
```

系统根据每行 4 列, 自动计算成 3 行 4 列的数组, 并按第 0 行、第 1 行、第 2 行顺序为各元素赋值。但注意表示列的[]中的常量不能省略。也可以用以下形式为二维数组中部分元素赋初值, 即

```
int x[][4]={{5}, {0}, {12, 20}};
```

形成的二维矩阵为:

```
5  0  0  0
0  0  0  0
12 20  0  0
```

例 3.2 求二维数组中每行元素的和并输出。

```
#include <iostream.h>
void main()
{ double data[3][4]={{1.2, 1.5, 1.7, 1.9},{2.0, 2.4, 2.6, 2.8},{3.2, 3.6, 3.7, 3.8}};
  double sum;
  for (int i=0; i<3; i++)
  { sum=0.0;
    for (int j=0; j<4; j++) sum+=data[i][j];
    cout<<" 第" <<i<<" 行和:" <<sum<<endl;
  }
}
```

运行结果为:

第 0 行和: 6.3

第 1 行和: 9.8

第 2 行和: 14.3

3.字符型数组的初始化

在 C++中常用字符数组处理字符串, 因此字符型数组赋初值时既可以赋字符常量, 也可以赋字符串。给字符型数组赋字符串时, 每个数组元素存放一个字符, 字符串的长度即为字符型数组中元素的个数。给字符型数组赋字符串时定义的长度要比实际字符个数多 1, 以存放字符串结束标志 ' \0' 。

以下几种初始化形式是等效的:

```
char ch[6]={' t' , ' o' , ' t' , ' a' , ' l' , ' \0' };
char ch[6]=" total" ;
char ch[]={ " total" };
char ch[]=" total" ;
```

一般一维字符型数组赋初值时常省略[]中表示数组大小的常量, 直接赋字符串, 编译系

统根据字符串的长度自动计算数组的大小。

字符型数组赋单个字符时，存放的是字符而不是字符串，如：

```
char d[]={ ' \t' , ' \t' , ' \a' , ' \a' , ' \n' };
```

这种方式通常用于在键盘上生成不可见字符。

为字符型数组初始化时要注意数组元素的个数与所赋字符串长度的关系，以免引起不可预料的错误。例如：

```
char array[6]=" program" ;
```

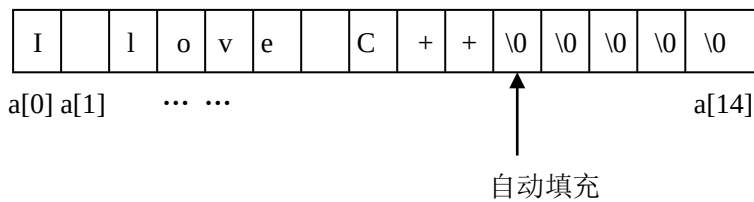
该定义不会产生编译错误，但由于改写了数组的大小，占用了数组空间以外的存储单元，容易引起混乱，甚至是危险的。这种情况最好写成：

```
char array[]=" program" ;
```

当字符型数组定义的空间大于初始化字符串长度时，剩余的空间填充 ' \0 ' ，例如：

```
char str[15]=" I love C++" ;
```

str 数组中存放的是：



二维字符型数组初始化时，可以用以下几种形式，结果是等效的。

```
char s1[3][5]={{ 'b','o','o','k','\0'},{ 'p','e','n','\0'},{ 'b','a','g','\0' }};
```

```
char s2[3][5]={" book" , " pen" , " bag" };
```

```
char s3[][5]={" book" , " pen" , " bag" };
```

```
char s3[][5]={{ " book" }, { " pen" }, { " bag" }};
```

例 3.3 统计一个字符串中字符的个数。

```
#include <iostream.h>
void main()
{ char str[80]=" C++ Programming Today" ;
  int count (0), i(0);
  while (str[i]!=' \0' )
  { count++; i++; }
  cout<<" 字符个数=" <<count<<endl;
}
```

运行结果为：

字符个数=21

3.1.3 数组的输入输出

定义数组后，在程序中就可以使用它。数组中的数据除在定义时初始化外，更多时候是通过赋值或输入输出得到。

例 3.4 向数组 a 中输入 10 个数，求其中的最大值和最小值。

```
#include <iostream.h>
void main()
{ int a[10], amax, amin;
  cout<<" 输入 10 个数:" ;
```

```

for(int i=0; i<10; i++) cin>>a[i];
i=0;
amax=amin=a[0];
while(i++<9)
{ if(a[i]>amax) amax=a[i];
  if(a[i]<amin) amin=a[i];
}
cout<<" 最大值=" <<amax<<endl;
cout<<" 最小值=" <<amin<<endl;
}

```

运行结果为:

```

输入 10 个数: 4 6 8 23 53 51 60 46 78 12
最大值=78
最小值=4

```

注意，二维数组的输入输出一般要用到二重循环，以分别控制二维数组的行和列。

例 3.5 向二维数组 array 中输入数据，输出该二维数组，并求 array 数组中所有元素的和。

```

#include <iostream.h>
void main()
{ int array[3][3];
  cout<<" 按行输入 3 行 3 列数组\n" ;
  for(int i=0; i<3; i++)
    for (int j=0; j<3; j++)
      cin>>array[i][j];          //按行输入二维数组
  cout<<" 按行输出数组\n" ;
  for(i=0; i<3; i++)
  { for(int j=0; j<3; j++) cout<<array[i][j]<<"   " ;
    cout<<endl;                  //输出每行后换行
  }
  int sum(0);
  for(i=0; i<3; i++)
    for(int j=0; j<3; j++)
      sum+=array[i][j];
  cout<<" 和=" <<sum<<endl;
}

```

在 C++ 中一般将字符串存储在字符数组中，因此字符数组的输入输出可以有两种方法，即逐个字符输入输出和将整个字符串输入输出。

逐个字符输入输出与数值型数组的输入输出相同，即用一变量控制数组的下标，根据下标的变化输入或输出单个字符。将整个字符串输入输出时，在 cin 和 cout 中只用数组名即可。

例 3.6 输入一串字符，统计其中小写字母 e 出现的次数，并将该字符串输出。

```

#include <iostream.h>
void main()
{ char str[80];

```

```

int i(0), ne(0);
cout<<" 输入一个字符串:" ;
cin>>str;                      //输入字符串
while (str[i]!=' \0' )
{  if(str[i]==' e' )  ne++;
    i++;
}
cout<<" 字符串:" <<str<<endl;//输出字符串
cout<<" e 字符个数:" <<ne<<endl;
}

```

运行结果为:

输入一个字符串: asanedereeghkex

字符串:asanedereeghkex

e 字符个数:5

字符串输入输出要注意以下几点。

①用 cin 和 cout 输入输出字符串时，只写数组名。

②用 cin 输入字符串时，当输入空格、制表符或回车时，输入结束，并在最后加一个 '\0' 字符。例如:

```

char str[10];
cin>>str;

```

执行时若从键盘输入:

I love C++ <回车>

str 数组的内容是:" I\0" 两个字符，输入的其他字符没有送到 str 数组。因此用 cin 输入字符串时要注意中间不能有空格或制表符。若输入含有空格或制表符的字符串时可以用第 5 章介绍的字符串输入函数 gets()或用第 10 章介绍的输入流的成员函数 get()、getline()等实现。

③用 cin 输入字符串时，输入字符的个数要小于字符数组定义时的长度。例如:

```

char str[5];
cin>>str;

```

若输入字符串:program，超过 str 数组的长度，会破坏其他数据，使用时要小心。

④用 cout 输出字符串时，遇到第一个 '\0' 输出结束，'\0' 不输出。一般形式是:

```

cout<<字符型数组名;

```

表示以字符串形式输出数组中的全部内容。此形式仅用于字符型数组，其他类型的数组不能用数组名作为输出，只能使用数组元素。

C++提供了一些使用方便、功能很强的处理字符串的函数，详见第 4 章。

3.1.4 数组应用

在程序设计中，许多问题的处理都是基于数组的。下面通过例题进一步了解和掌握数组的使用方法。

例 3.7 用冒泡法对 data 数组中的 10 个数由小到大进行排序。

冒泡排序(bubble sort)方法的基本过程是:从数组第 j=0 个元素开始，每两个相邻元素值进行比较，即 data[0]和 data[1]进行比较，data[1]和 data[2]进行比较，……，若 data[j]>data[j+1]则进行交换，使较大的值一直往后移。如此进行一遍后，最大的数下沉到底部，成为数组中最后一个元素，而小的数上升，像气泡一样。

再从头开始，进行第二轮比较，次大的数下沉。重复以上操作，直到排序完成。

程序中第 1 个 for 循环用于输入数组中的 N 个数。

排序过程用嵌套的 for 循环实现。外层的 for 循环每执行一次，内层循环从第 0 个元素开始与相邻元素进行比较，有满足 `data[j]>data[j+1]` 条件的，则进行交换。内层循环每次执行 0~N-i-1 次。

程序如下：

```
#include <iostream.h>
void main()
{   int data [10];
    int i,j,temp;
    cout<<" 输入 10 个整数:" ;
    for (i=0; i<10; i++) cin>>data[i];
    for(i=0; i<10; i++)
    {   for(j=0; j<10-i-1; j++)
        if (data[j]>data[j+1])
        {   temp=data[j+1];           //两个数进行交换
            data[j+1]=data[j];
            data[j]=temp;   }
    }
    cout<<" 排序结果:" ;
    for(i=0; i<10; i++) cout<<data[i]<<"   " ;
    cout<<endl;
}
```

运行结果为:

输入 10 个整数: 2 5 8 4 12 63 18 73 52 31

排序结果: 2 4 5 8 12 18 31 52 63 73

例 3.8 求二维数组 b 中每行元素的最大值以及最大值所在的位置。

```
#include <iostream.h>
void main()
{   int  b[4][5], bmax, row, col;
    cout<<" 按行输入 4 行 5 列数组:\n" ;
    for(int i=0; i<4; i++)
        for (int j=0; j<5; j++)
            cin>>b[i][j];
    for(i=0; i<4; i++)
    {   bmax=b[i][0];
        row=i;           //row 记住行号
        col=0;
        for(int j=1; j<5; j++)
            if (b[i][j]>bmax) { bmax=b[i][j]; col=j; } //求每行的最大值, col 记下列号
        cout<<row<<"行"<<col<<"列"<<" "<<"最大值="<<bmax<<endl;
    }
}
```

运行结果为:

按行输入 4 行 5 列数组:


```

2  4  7  0  6
3  5  9-1 23
45  7  9  2 -2
0  5  12 3  6
0行2列  最大值=7
1行4列  最大值=23
2行0列  最大值=45
3行2列  最大值=12

```

例 3.9 输入一个字符串存入数组 `a` 中，对该字符串中的每个字符用+3 的方法进行加密后送入数组 `am`，输出加密后的结果。

```

#include <iostream.h>
void main()
{ char a[80], am[80];
  int i=0;
  cout<<" 输入一串字符:" ; cin>>a;
  while(a[i]!=' \0' ){ am[i]=a[i]+3; i++; }
  am[i]=' \0' ;
  cout<<" 加密后结果:" <<am<<endl;
}

```

运行结果为:

```

输入一串字符: string
加密后结果: vwulqj

```

3.2 指针

指针也是一种数据类型。具有指针类型的变量，称为指针变量。与前面介绍的其他变量不同，指针变量中存放的不是数据，而是内存单元的地址，通过这个地址可以访问其中的内容。

当运行一个程序时，程序中用到的数据都要保存到计算机的内部存储器中。计算机的内部存储器是由很多个存储单元组成的，每个存储单元都有一个编号，这个编号称为地址。计算机通过这个地址来读写单元中的数据。

程序中定义的一般变量要在内存中分配相应的存储单元。如果在程序中给某个变量赋值，实际上就是将这个值存放到该变量的内存单元中，变量名就是这个单元的名称。数据类型不同，每个变量分配的单元数目不同。例如，字符型变量占用一个内存单元，`float` 浮点型变量占用 4 个存储单元。

存储单元既可以存放数据，也可以存放地址。可以将一个变量的地址保存到某个存储单元中。存放变量地址的内存单元称为指针变量，简称指针。

3.2.1 指针变量的声明和使用

1. 指针变量的声明

指针是个变量，使用指针变量之前必须声明。声明指针变量的形式为:

```
类型名 *标识符;
```

例如:

```
int *pi;
```

```
char *pc;
double *pd;
```

声明了三个指针变量，其中 `pi` 存储一个整型变量的地址；`pc` 存储一个字符型变量的地址；`pd` 存储一个双浮点型变量的地址。指针变量前的“*”为声明指针变量的说明符，而不是指针变量的一部分，只是说明 `pi`、`pc` 和 `pd` 等标识符是指针变量名，而不是普通变量名。指针也有类型，指针的类型就是它所保存的地址中所存储的数据的类型。

在 C++ 中各种数据类型都可以指定指针变量。指针变量的作用域与一般变量相同。指针除了可以指向变量外，还可以指向其他任何数据结构，如数组、结构体、联合体等，又可以指向函数。

2. 指针变量的使用

与指针变量有关的两个运算符是：

```
&    取变量地址运算符
*    取指针变量所指对象内容的运算符
```

其中 `&` 仅对一般变量使用，表示取该变量的地址。“*”放在指针变量名前，表示取指针变量所指单元的内容。例如：

```
int a, *p1;
p1=&a;      // 将变量 a 的地址赋给指针 p1
*p1=25;     // 将 25 赋给指针 p1 所指单元中
```

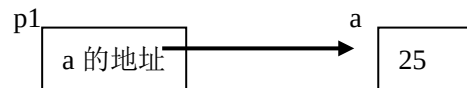


图 3.1 指针变量

`p1=&a` 执行后 `p1` 中的内容是变量 `a` 的地址，习惯上称 `p1` 指向 `a`。而 `*p1=25` 表示将 25 赋给 `p1` 所指的单元，即将 25 存入变量 `a` 中，如图 3.1 所示。

注意，语句 `*p1=25` 中的“*”和声明指针变量名前的“*”不同。程序中出现在指针变量名前的“*”是运算符，表示取指针所指内容。而声明时出现的“*”仅表示“*”后的某标识符定义为指针变量名。

指针变量声明后，在使用前必须给它赋一个合法的值，可以在程序中赋值，也可以在声明时对指针进行初始化，例如：

```
int x=50;
int *px=&x;
double *py=0;
```

`px` 指针指向整型变量 `x`。在声明 `py` 指针时初始化为 0，值为 0 的指针叫空指针，空指针表示不指向任何地方。另外注意，对指针赋的值必须是同一种类型变量的地址。如果将一个变量的地址赋予另一种不同类型的指针，会产生语法错误，例如：

```
int a, b;
double *px=&a;    ?//出错，指针与所指变量类型不同
```

因为指针也是个变量，程序中将哪个变量的地址赋给指针，指针就指向哪个变量。例如：

```
int a, b;
int *p1;
p1=&a;      //p1 指向变量 a
*p1=100;   //将 100 存入 a
```

```

cout<<*p1<<endl;      //输出 p1 指针所指单元的内容，即 a 中的内容 100
p1=&b;                 //p1 指向变量 b
*p1=200;               //将 200 存入 b
cout<<*p1<<endl;      //输出 p1 指针所指单元的内容，即 b 中的内容 200 :

```

需要说明，指针只能接受地址量，如果将一个任意常数赋予指针，会产生语法错误。
同类型的指针之间可以互相赋值，例如：

```

double  x=6.78, y=3.45;
double  *p1=&x, *p2=&y;
p1=p2;                //执行后 p1 和 p2 都指向变量 y

```

执行 `p1=p2` 前后见示意图 3.2。

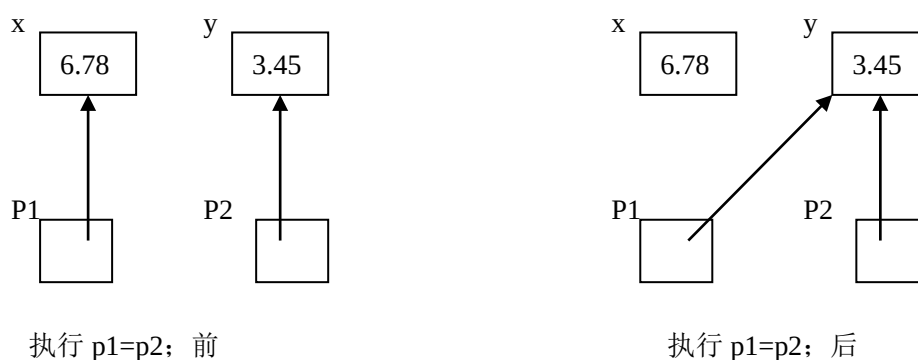


图 3. 2 同类型指针赋值

例 3.10 指针使用示例。

```

#include <iostream.h>
void main()
{
    int a=10;
    int *p1,*p2=&a;
    p1=p2;
    cout<<" a=" <<a<<endl;
    cout<<" *p1=" <<*p1<<endl;
    cout<<" *p2=" <<*p2<<endl;
    cout<<a+(*p1)+(*p2)<<endl;
    *p1=20;
    cout<<" a=" <<a<<endl;
    cout<<p1<<" ?" <<p2<<" ?" <<&a<<endl;
}

```

运行结果为:

```

a=10
*p1=10
*p2=10
30
a=20
0x0065FDF4  ?0x0065FDF4  0x0065FDF4

```

最后一个 `cout` 输出的是三个用 16 进制表示的相同的地址，即为变量 `a` 的地址。不同环

境下此处输出结果不同。这里只是验证一下该程序中 p1 和 p2 都是存放 a 的地址。实际在程序设计中具体的地址值是什么程序设计人员不需要关心,我们关心的是指针与所指变量之间的关系。

3.2.2 指针的运算

指针变量可以参加运算。设 a 为变量, p、q 为同一类型指针, 指针允许的运算包括:

&a	//取变量地址
*p	//取指针所指内容
p=q	//指针间赋值
p++、p--或++p、--p	//指针增 1 或减 1
p+n 或 p-n	//指针加上或减去一个整数 n
p-q	//指针相减
p<=q	//指针关系运算

其中指针的增减 1、指针与一个整数的加减运算、指针相减及指针的关系运算只有与数组结合起来使用才有实际意义。前面介绍数组时曾强调, 数组元素在计算机内存中占用一片连续的存储单元, 这就意味着这片连续的存储单元的地址是连续编号的。假设一片连续存储单元存放多个数据, 指针指向其中的某个单元, 那么指针加 1、减 1 运算是指针由当前位置向前或向后移动一个数据单元位置; 指针加或减一个整数 n, 是指针由当前位置向前或向后移动 n 个数据单元位置; 当两个指针指向同一数组的不同元素时, 指针相减的结果即是两指针所指位置间的数据的个数; 两个指向同一数组的指针进行关系运算时, 表示的是它们所指单元的先后位置。

3.3 指针与数组

在 C++ 中指针与数组有着密切的关系, 任何由数组下标完成的操作都可以用指针实现。

3.3.1 一维数组与指针

例如有如下定义:

```
int array[5]={ 1, 3, 5, 7, 9};
```

编译程序为该数组分配 5 个连续的整型量空间以存放 5 个元素, 同时数组名作为该数组的首地址使用, 即 array[0] 元素的地址, 见图 3.3 所示。因此 array 和 &array[0] 是相等的。array 是指向 array[0] 元素的指针, 那么 array+1 是第 1 个元素地址, array+2 是第 2 个元素的地址, …… , array+i 是第 i 个元素的地址。因此 *array 表示的就是元素 array[0] 的值, *(array+1) 表示的就是元素 array[1] 的值, *(array+i) 表示的就是第 i 个元素的值。

程序中可以用 array[i] 表示数组中的第 i 个元素, 也可以用 *(array+i) 表示第 i 个元素。第 i 个元素的地址既可以用 &array[i] 表示, 也可以用 array+i 表示。

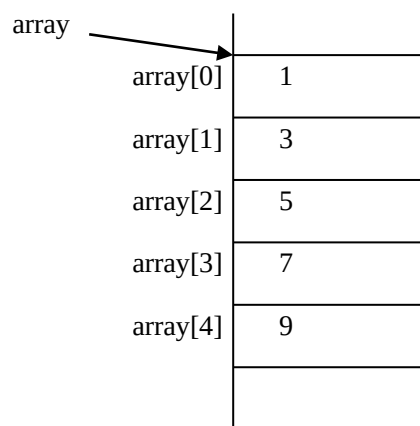


图 3.3 指针与数组

例 3.11 用数组名访问数组元素，求 a 数组中元素之和。

```
#include <iostream.h>
void main()
{   int a[5],i,sum1,sum2;
    cout<<" 输入 5 个数:" ;
    for(i=0; i<5; i++) cin>>*(a+i);           //与 cin>>a[i]相同
    for(i=0,sum1=0;i<5;i++) sum1+=*(a+i);      //用数组名访问数组元素
    for(i=0,sum2=0;i<5;i++) sum2+=a[i];        //用下标访问数组元素
    cout<<" sum1=" <<sum1<<endl;
    cout<<" sum2=" <<sum2<<endl;
}
```

运行结果为:

输入 5 个数:3 6 20 45 2

sum1=76

sum2=76

从该例题可以看出，用数组名访问数组元素与用数组下标的效果相同。

在程序中可以声明一个与数组类型相同的指针，然后让这个指针指向这个数组，通过这个指针访问数组中的元素。

例如有以下语句:

```
int a[10], *p;
p=a;
```

此语句把数组 a 的首地址给指针 p，p 即为指向 a 数组的指针。因此，p+1 指向 p 的下一个存储单元，即 a[0]的下一个元素 a[1];p+i 指向 p 后的第 i 个元素，也就是 a[i]。可以通过 p 访问数组 a 中的元素。此时表示一维数组 a 第 i 个元素的地址可以用以下几种形式:

```
&a[i]
a+i
p+i
```

若表示一维数组 a 中第 i 个元素，可以用以下几种形式:

```
a[i]
*(a+i)
*(p+i)
```

p[i]

用指针去访问数组中的元素，执行效率更高。

例 3.12 利用数组名和指向数组的指针访问数组中的元素。

```
#include <iostream.h>
void main()
{   int a[5]={10,20,30,40,50};
    int *p=a;
    for(int i=0; i<5; i++) cout<<a[i]<<"    ";
    cout<<endl;
    for(i=0; i<5; i++) cout<<*(a+i)<<"    ";
    cout<<endl;
    for(i=0; i<5; i++) cout<<*(p+i)<<"    ";
    cout<<endl;
    for(i=0; i<5; i++) cout<<p[i]<<"    ";
    cout<<endl;
    p=a+3;
    cout<<*p<<endl;
    cout<<p-a<<endl;
}
```

运行结果为:

```
10  20  30  40  50
10  20  30  40  50
10  20  30  40  50
10  20  30  40  50
40
3
```

执行结果中倒数第 2 行的 40 为执行 `p=a+3` 后指针 `p` 所指的内容，即 `a` 数组中由首地址开始的后面第 3 个元素的内容。执行结果中最后的 3 为 `p-a` 的值。两地址相减结果为 `p` 指针与 `a` 数组首地址之间元素的个数。

用指针处理数组时需要注意以下几点。

①数组名是个地址常量，而指针是个地址变量。编译时数组名作为数组的首地址已确定，程序运行时其值是固定不变的。因此若定义

```
int a[10];
```

后进行 `a++` 或 `a=p` 等运算都是错误的。而指针是个地址变量，将哪个数组名赋给它，它即指向哪个数组。例如:

```
int a[5], b[10], *p;
p=a;                //p 指向 a 数组首地址
p=b;                //p 指向 b 数组首地址
p++;                //p 指向数组中的下一个元素
```

②当指针运算符与其他运算符联合使用时，要注意运算符的优先级别，例如:

```
*p++                //取 p 所指内容后，p 指针加 1
*++p//p 指针加 1 后所指的内容
(*p)++//将所指向的单元内容加 1
```

③用指针处理数组时，指针移动的值不要超过数组元素下标定义的范围，否则越界。

越界后得到的结果可能毫无意义。

3.3.2 多维数组与指针

在 C++ 中二维数组可以作为一维数组来处理，例如有以下定义：

```
int a[3][4];
```

可以理解为 a 数组是由三个元素组成的一维数组，这三个元素的名字为 a[0]、a[1]、a[2]，其中的每个元素又是由四个元素组成的一维数组。因此，a 数组可以分解为：

$$a \left\{ \begin{array}{l} a[0] \rightarrow a[0][0] \quad a[0][1] \quad a[0][2] \quad a[0][3] \\ a[1] \rightarrow a[1][0] \quad a[1][1] \quad a[1][2] \quad a[1][3] \\ a[2] \rightarrow a[2][0] \quad a[2][1] \quad a[2][2] \quad a[2][3] \end{array} \right.$$

与处理一维数组一样，数组名是指向数组的首地址。因此 a 是三个一维数组 a[0]、a[1]、a[2] 的首地址。而 a[0]、a[1]、a[2] 又是三个数组名，分别表示三个一维数组的首地址。因此表示二维数组 a 中第 i 行第 j 列元素的值可以用以下形式：

$*(*(a+i)+j)$

$*(a[i]+j)$

a[i][j]

需要说明的是：在程序中可以用 a[i] 表示二维数组 a 中第 i 行的首地址。a[i] 是个一维数组名。二维数组中的 a[i] 本身并不占实际的内存单元，也不存放某个元素的值，只是一种计算地址的表示方法。

在程序设计中既可以用下标法引用数组元素，也可以用指针(地址)法引用数组元素。

例 3.13 使用指针法访问二维数组。

```
#include <iostream.h>
void main()
{ int a[3][4]={1, 2, 3, 4},{5, 6, 7, 8},{9, 10, 11, 12}};
  cout<<*(a+0)<<*<a[0]<<a[0][0]<<endl;
  cout<<*(a+1)+0)<<*(a[1]+0)<<a[1][0]<<endl;
  cout<<*(a+2)+1)<<*(a[2]+1)<<a[2][1]<<endl;
}
```

运行结果为：

```
1 1 1
5 5 5
10 10 10
```

在程序设计中，可以将二维数组名赋给一个指针，根据二维数组在机器内连续存放的方式，再用这个指针处理二维数组。

例 3.14 将输入的年、月、日转换为本年度的第几天。

```
#include <iostream.h>
int date[][13]={0,31,28,31,30,31,30,31,31,30,31,30,31},
               {0,31,29,31,30,31,30,31,31,30,31,30,31}};
void main()
{ int y, m, d, yd;
  int *p, i, *j;
  cout<<" 输入年、月、日：" ; cin>>y>>m>>d;
  p=date[0];
  i=(y%4==0 && y%100!=0||y%400==0); //判 y 是平年还是闰年
```

```

yd=d;
if(i) p+=13;           //若 i 为 1，是闰年
j=p+m;
while(p<j) { yd+=*p; p++; }
cout<<y<<" 年" <<m<<" 月" <<d<<" 日是该年的第" ;
cout<<y<<endl;
}

```

运行结果为:

输入年、月、日:2005 5 1

2005 年 5 月 1 日是该年的第 121

对例 3.14 程序说明如下:

①数组 `date` 中第 0 行存放平年 12 个月的天数，第一行存放闰年 12 月的天数。

②当输入年、月、日给 `y`、`m`、`d` 变量后，判 `y` 是平年还是闰年。当条件满足时，是闰年，`i=1`;否则 `i=0`。

③当 `i=0` 时，说明是平年，此时 `p+=13` 不执行，直接执行 `j=p+m`，使 `j` 指针指向 `m` 月，执行 `while` 语句，用 `p` 指针处理 `date` 数组第 0 行的元素，`p` 每加 1，指向下一个月，将各月份相加，用 `yd+=*p` 求出该年的 `m` 月 `d` 日是这年的第几天。当 `i=1` 时，说明是闰年，执行 `p+=13` 跳过第 0 行的 12 个月数，直接指向 `date` 数组的第一行，再通过 `p` 加 1 处理第一行中的每个月。

这个题利用二维数组按行存放，相邻两行的地址首尾相接，用一个指针指向连续存放的二维数组，通过指针的移动访问数组中的元素。

对于二维数组还可以使用行指针进行处理。行指针定义形式如下:

类型名 (*指针名)[列长度];

其中，列长度是指该指针所指向的二维数组的列长度。例如，要定义行指针 `px` 并指向二维数组的首行，可写为:

```
int x[3][5], (*px)[5]=x;
```

行指针加 1 表示指向二维数组的下一行。使用行指针可以按行处理二维数组。下面是使用行指针计算二维数组各行的平均值的程序:

```

#include <iostream.h>
const int Ncol=5;
void main()
{ int x[3][Ncol]={ {4, 9, 6, 12, -3}, {8, 7, 10, 5, 20}, {17,1,32,13,24}};
  int (*px)[Ncol]=x;
  int s,i,j;
  for(i=0; i<3; i++, px++)
  { for(j=0, s=0; j<Ncol; j++) s+=(*px+j);
    cout<<i<<" " <<s/(float)Ncol<<endl;
  }
}

```

运行结果为:

0 5.6

1 10

2 17.4

程序中 `*(px+j)` 表示取 `px` 所指向行的第 `j` 个元素值。

3.3.3 字符型指针与字符型数组

在 C++ 中字符串用字符型数组或字符型指针处理。

例 3.15 字符型指针与字符型数组。

```
#include <iostream.h>
void main()
{ char str1[]=" character array" ;
  char *ps=" character point" ;
  cout<<str1<<endl;
  cout<<ps<<endl;
}
```

运行结果为:

```
character array
character point
```

将一个字符串赋给一个字符型指针时,该指针指向字符串的首地址,即第一个字符的地址。可以通过移动该指针访问字符串中的字符。

字符型数组和字符型指针有以下区别:

①字符型数组只能在初始化时为整体赋值,不能在程序中向数组名赋值。而字符型指针除初始化外,还可以在程序中赋字符串。例如:

```
char s[10], *p;
s=" abcdefg" ;           //出错, s 为地址常量, 不能出现在 "=" 左侧
p=" abcdexy" ;           //正确, p 为字符型指针变量, 可以出现在 "=" 左侧
```

②对于字符型数组,编译程序根据定义数组时的大小分配存储空间。而对于字符型指针,仅给该指针变量分配存放一个地址的单元。该单元可以存放任一字符串的首地址。

例 3.16 实现两字符串的连接。

```
#include <iostream.h>
void main()
{ char s1[20]=" abcde" ,*ps1;
  char s2[]=" xyzf" ;
  int i=0;
  ps1=s1;
  while (*ps1) ps1++;           //寻找 s1 字符串的结束位置
  int j=0;
  while(*(s2+j))
  { *ps1=*(s2+j); ps1++; j++; } //将 s2 字符串逐个复制到 s1 后
  *ps1='\0' ;
  cout<<s1<<endl;
}
```

运行结果为:

```
abcdexyzf
```

3.4 指针数组

所谓指针数组即数组中的每个元素都是指针变量,或称由指针构成的数组。指针数组中

的每个指针具有相同的数据类型。与普通数组一样，使用指针数组前必须先定义。定义形式为：

数据类型*数组名[元素个数];

例如：

```
int *px[10];
```

由于[]的优先级别比“*”高，所以px首先是个数组名，该数组有10个元素，再与前边的“*”结合，每个元素是个指针，每个指针指向一个整型数。又如：

```
char *ps[5];
```

ps是个字符型的指针数组，ps数组中每一个元素都是一个指向字符串的指针。用字符型的指针数组可以处理长度不同的字符串。

指针数组在定义的同时也可以初始化。因为指针数组中的每个元素是个指针，所以初始化的值应是变量的地址。例如：

```
int a, b, c;
```

```
int *p[3]={&a, &b, &c};
```

例 3.17 对数组a中每个元素求平方值，用指针数组实现。

```
#include <iostream.h>
void main()
{ int a[5]={1,2,3,4,5};
  int *p[]={&a[0], &a[1], &a[2], &a[3], &a[4]};
  for(int i=0; i<5; i++){ *p[i]*=*p[i]; cout<<*p[i]<<" "; }
  cout<<endl;
}
```

字符型指针数组可以用以下形式初始化：

```
char s1[]=" Basic" ;
char s2[]=" Fortran" ;
char s3[]=" C++" ;
char s4[]=" Java" ;
char *ps[]={s1, s2, s3, s4}; //ps 为指针数组
```

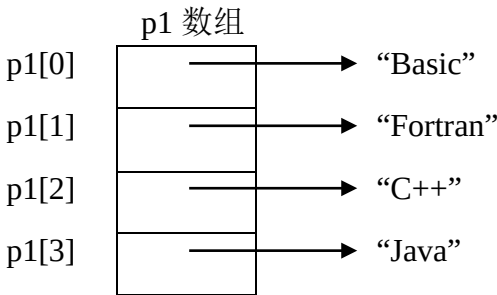


图 3.4 字符型指针数组与所赋字符串的关系

也可以直接给字符型指针数组赋字符串。例如：

```
char *p1[]={ " Basic" , " Fortran" , " C++" , " Java" };
```

p1 数组中存放 4 个字符串的初始地址，其关系见图 3.4。

例 3.18 用字符型指针数组编写程序。当输入 1~12 中某个值时，输出对应的月份。

```
#include <iostream.h>
```

```

void main()
{ char*month[]={“illegal month”,“January”,“February”,“march”,“april”,“may”,“june”,
                “july”,“august”,“September”,“october”,“November”,“December”};
  char *find;
  int n;
  cout<<“ 输入月份(1~12): ” ; cin>>n;
  find=(n<1||n>12)?month[0]: month[n];
  cout<<n<<“ —— ” <<find<<endl;
}

```

第一次执行程序:

输入月份(1~12): 8

8——august

第二次执行程序:

输入月份(1~12): 13

13——illegal month

例 3.19 用指针数组处理二维数组。

```

#include <iostream.h>
int x[3][3]={ {1,2,3},{4,5,6},{7,8,9}};
int *p[3]={x[0], x[1], x[2]};
void main()
{ for(int i=0; i<3; i++)
  cout<<*p[i]<<“ ” <<*(p[i]+2)<<endl;
}

```

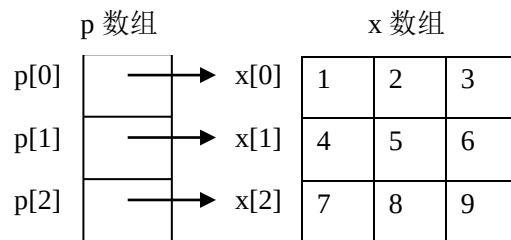


图 3.5 指针数组 p 与 x 数组的关系

该程序中指针数组 p 与 x 数组的关系见图 3.5。

p 数组中每个元素存放 x 数组每行的首地址，输出 *p[i]即为 x[i][0]的内容，*(p[i]+2)即为 x[i][2]元素值。

运行结果为:

```

1 3
4 6
7 9

```

3.5 堆内存分配

程序运行时需要的数据空间通常是在程序设计时通过定义变量或数组的方式由系统预

先分配。但在设计程序时,有些数据空间的大小还不能确定,只有在程序运行过程中才能确定,使用堆(heap)内存可以实现这一功能。堆是一种内存空间。它允许程序运行时根据需要申请内存空间。由于这种空间的大小在编译和连接时并不确定,而是随着程序运行可大可小,所以堆内存是动态的,又称动态内存分配。

C++的动态内存分配技术使用 `new` 和 `delete` 两个运算符,其中 `new` 用于申请一块动态空间。当不需要用 `new` 申请的空间时,可以用 `delete` 释放。

1.new 运算符

`new` 运算符的使用形式为:

指针=new 数据类型名

作用是从内存的动态区域申请指定数据类型所需的存储单元。若分配成功,该存储单元的首地址赋给指针;否则,指针得到一个空地址。其中指针所指的数据类型要和 `new` 后边给定的数据类型相同。例如:

```
double *p;
p=new double;
```

表示申请一个 `double` 型数据的存储单元。申请成功后, `p` 中存放该单元的地址,在程序中可以通过 `p` 访问该单元。

与其他变量一样,在动态申请内存单元时,也可以同时对该单元初始化。例如:

```
double *p;
p=new double(12.3576);
```

为申请的内存单元指定初始值为 12.3576。

同样,也可以用运算符 `new` 在堆中申请一块连续的存储空间,创建一个数组。动态创建一维数组的形式为:

指针=new 数据类型[元素个数]

其中元素个数可以是变量。数据类型为要创建的数组的类型。该类型要和“=”号前指针的类型相同。

如果创建数组成功,指针中存放数组的首地址,否则指针是空指针。前面介绍过,数组名表示数组的初始地址,因此在程序中该指针可以作为数组名使用。

但要注意,动态为数组分配内存空间时不能进行初始化。

例如:

```
int size, *ip;
cin>>size;
ip=new int[size];           ?//创建 size 个元素的整型数组
char *sp=new char[20];     ?//创建 20 个字符的字符数组
```

用 `new` 可以创建多维数组时也不能初始化。创建二维数组的形式是:

指针=new 数据类型[行数][列数]

用 `new` 也可以动态创建对象或对象数组。

2.delete 运算符

当程序中不再使用用 `new` 创建的存储单元时,必须用 `delete` 运算符释放。它的使用形式为:

```
deletd 指针名;           //释放指针所指内存单元
deletd []指针名;        ?//释放数组内存单元
```

`delete` 运算符并不删除指针,必要时可以对指针重新赋值。例如:

```
int *pi=new int(5);
delete pi;
```

```
pi=new int;
```

注意:对于指向动态分配内存单元的指针, 在其所指向的内存单元没有释放前, 该指针不能重新赋值。例如:

```
int *pi=new int;
```

```
*pi=5;
```

```
pi=new int;
```

?//出错

还要注意, 对每个 new 运算符创建的内存单元, 只能用 delete 释放一次。如果用 delete 运算符连续多次释放, 将会产生一个运行期间的系统错误。

用 delete 可以对内容为空的指针进行多次连续释放。例如:

```
int *p;
```

```
p=new int;
```

```
delete p; delete p;
```

//第二次释放将产生运行错误

但若为

```
int *p=NULL;
```

```
delete p;delete p;
```

则是安全的。其中的 NULL 是空指针, 它表示指针不指向任何地方, 常常被用来赋给指针或判断指针是否为空指针。NULL 的值会因为使用的系统不同而取不同的值。一般 PC 机上的编译环境中, NULL 的值是 0。

例 3.20 计算一批数据的算术平均值。

```
#include <iostream.h>
```

```
#include <stdlib.h>
```

```
void main()
```

```
{ int m;
```

```
cout<<" 输入数据个数:" ; cin>>m;
```

```
int *p;
```

```
p=new int[m];
```

//动态分配内存单元

```
if(p==NULL)
```

```
{ cout<<" 动态内存分配失败\n" ; exit(1); } //若分配不成功, 则退出程序
```

```
cout<<" 输入待处理数据:\n" ;
```

```
for( int i=0; i<m; i++) cin>>p[i];
```

```
float s=0.0;
```

```
for(i=0; i<m; i++) s+= p[i];
```

```
cout<<" 平均值=" <s/m<<endl;
```

```
delete []p;
```

```
}
```

运行结果为:

输入数据个数: 5

输入待处理数据:

10 20 30 40 50

平均值=30

程序中 exit(1)函数用来强行终止程序的执行, 返回操作系统。它在 stdlib.h 文件中定义。使用时, 若在括号中写 0, 表示程序正常终止;写非 0 时, 表示程序非正常终止。

3.6 const 指针和 const 引用

1.const 指针

用关键字 `const` 修饰指针时，`const` 的位置不同，含义也不同，可以有以下几种情况。

①在声明指针语句的类型前边加 `const`，表示是一个指向常量的指针。在程序中不能通过指针改变它所指向的常量的值，但指针本身的值可以改变。例如：

```
char *p1=" Hello" ;
char *p2=" ok!" ;;
const char *pc=p1;      //声明指向 char 型常量的指针
*pc='Y';                //错，不能改变指针所指向的值
pc=p2;                  //正确，指针本身的值可以改变
```

②在声明指针的“*”和指针名之间加 `const`，表示声明一个指针常量，或称常指针。此时指针本身的值不能改变，但它指向的数据的值可以改变。另外注意，在声明指针常量时必须初始化。例如：

```
char *p=" ok" ;
char *const pc=" Hello" ;    //声明指针常量
pc=p;                        //错，指针本身的值不能修改
pc=" Yes" ;                  //错，改变指针的值
*pc=' h' ;                   //正确，所指向的值改变
*(pc+2)=' k' ;               //正确，所指向的值改变
int *const p2;               //错，声明指针常量时必须初始化
```

③关键字 `const` 在上述两个地方都加表示声明一个指向常量的指针常量。因此指针本身的值和它所指向的数据的值都不能改变，同时在声明时也必须进行初始化。例如：

```
int a=5, b=6;
const int * const pa=&a;
pa=&b;                        //错
*pa=7;                        //错
a=7;                          //正确
```

2. const 引用

使用 `const` 也可以修饰引用，被修饰的引用称为常引用。常引用所引用的对象不能改变。

例如：

```
int a=14;
const int& vfa=a;            //声明 vfa 为常引用
vfa=20;                      //错
```

在程序设计中，常指针和常引用往往用来做函数的参数。具体使用见第 4 章函数。用关键字 `const` 也可以修饰对象和类中的成员。

3.7 例题

例 3.22 任意输入一个整数，求出它的所有因子并存入一个数组。

```
#include <iostream.h>
void main()
{ int number, factor[20];
  int k=0;
```

```

cout<<" 输入一个整数:" ; cin>>number;
for(int i=1; i<number; i++)
    if(number%i==0) { factor[k]=i; k++; }
cout<<" 数" <<number<<" 的因子是:" ;
for(i=0; i<=k-1; i++) cout<<factor[i]<<"    " ;
cout<<endl;
}

```

运行结果为:

输入一个整数:36

数 36 的因子是:1 2 3 4 6 9 12 18

例 3.23 输入 10 个数，将其中最小的数与第一个数对换，将其中最大的数与最后一个数交换。用指针实现。

```

#include <iostream.h>
void main()
{ int number[10];
  int *max,*min,*p, temp;
  cout<<" 输入 10 个数" ;
  for (int i=0;i<10;i++) cin>>number[i];
  max=min=number;
  for (p=number+1; p<number+10; p++)
      if (*p>*max) max=p; // 将大数地址赋给 max
      else if (*p<*min) min=p; // 将小数地址赋给 min
  temp=number[0]; number[0]=*min; *min=temp; // 将最小数与第一个数交换
  if(max==number) max=min;
  temp=number[9]; number[9]=*max; *max=temp; // 最大数与最后一个数交换
  cout<<" 交换后的数组:" ;
  for (p=number; p<number+10; p++) cout<<*p<<"    " ;
  cout<<endl;
}

```

运行结果为:

输入 10 个数:5 7 9 23 26 54 16 2 60 10

交换后的数组:2 7 9 23 26 54 16 5 10 60

练习 3

1.回答下列问题:

- (1)什么是数组，如何定义数组？
- (2)字符型数组有哪几种初始化形式？
- (3)什么是指针，指针的类型指什么？
- (4)举例说明指针的两个运算符&和*的作用是什么？
- (5)指针有哪些运算，指针的加减运算一般在什么情况下使用？
- (6)字符型数组和字符型指针处理字符串有何区别？
- (7)如何用指针来表示一、二维数组元素的值和元素的地址？
- (8)什么是引用，它和指针有什么区别？
- (9)什么是指针数组，指针数组与一般数组有什么区别？
- (10)用 new 和 delete 创建和释放动态数组的形式是怎样的？

2.阅读下列程序并写出运行结果:

(1)#include <iostream.h>

```
void main()
{   int a[10]={ 2,5,8,-3,-6,9,12,10};
    int i=1;
    while (i<9)
    { cout<<a[i-1]<<"    " <<a[i+1]<<endl;
      i+=2;
    }
    cout<<i<<endl;
}
```

(2)#include <iostream.h>

```
void main()
{ int bx[]={1,3,5,7,9,11,13,15};
  for(int *pb=&bx[7]; pb>=bx;pb--) cout<<*pb<<"    " ;
  cout<<endl;
}
```

(3)#include <iostream.h>

```
int main()
{ char str[]=" abcdefghijklmnopqrstuvwxyz " ;
  char *ps=" ***** " ;
  for(int i=1; i<3; i++)
  { cout<<ps<<endl;
    for(int j=0; j<26; j++) cout<<*(str+j)<<"    " ;
    cout<<endl;
  }
  return 1;
}
```

(4)#include <iostream.h>

```
void main()
{ int *p1;
```



```

double *p2;
p1=new int(4);
p2=new double[3];
p2[0]=1.5; p2[1]=2.5; p2[2]=3.5;
for(int n=0; n<3; n++)
    cout<<p2[n]*(p1)<<endl;
}

```

3.编写下列程序:

- (1)向数组 a 中输入 10 个整数，求其中的最大值、最小值和 10 个数的算术平均值。
- (2)输入 10 个学生一门课的成绩，分别统计大于平均值的人数和小于 60 分的人数。
- (3)任意输入一个字符串，统计其中英文字母“a”和“i”的个数，并将该字符串输出。
- (4)输入一个 3×3 的二维数组，分别求主对角线元素的和及次对角线元素的和。
- (5)设整型数组 a 中按序存放有以下数据:2、4、5、8、12、14、16、18、20、30。从键盘任意输入一个整数插入 a 数组，插入后该数组仍有序。输出插入后数组 a 中的内容。
- (6)求一个 5 行 3 列二维数组每行元素的和，并将求出的和按由小到大的次序排序后输出。
- (7)设字符型数组 str 和字符型指针 ps 中分别存放长度相同、内容不同的字符串，编程实现将两个字符串中的内容交换，输出交换前后的字符串。